

Faculty of Computing

Year 2 Semester 1 (2026)

IT2011 - Artificial Intelligence and Machine Learning

Lab Sheet 01

Learning Objectives

1. *Understand the difference between traditional programming and machine learning approaches.*
 - *Develop and implement a rule-based program.*
 - *Build, train, and assess a basic machine learning model.*
 2. *Investigate Generative AI concepts and practice prompt engineering.*
-

Introduction

This lab introduces Artificial Intelligence (AI) and Machine Learning (ML) through practical exercises. It covers traditional programming, its limitations, how ML overcomes these challenges, essential tools and techniques used in ML programming and the basics of prompt engineering.

Traditional programming involves writing **explicit rules** to process inputs and produce outputs. A programmer defines precise instructions for the computer to follow.

- For example, to determine if a student's grade is a pass or fail based on a score (passing if 50 or above), a rule checks the score against this threshold.

Below is a Python program implementing this logic:

```
# Program to check if a student passes or fails based on score
score = int(input("Enter the student's score (0-100): ")) # Convert user input to integer
if score >= 50: # Check if score is 50 or higher
    print("Pass") # Output for passing score
else:
    print("Fail") # Output for failing score
```

This approach is effective for simple, well-defined problems but struggles with complex tasks where rules are hard to define. For instance, traditional programming cannot easily handle tasks like

- recognizing faces in photos
- classifying emails as spam
- predicting weather patterns

These tasks involve intricate patterns, large datasets, or unpredictable inputs, making it impractical to write explicit rules for all possible cases.

Machine Learning (ML) addresses these limitations by allowing computers to learn patterns from data without explicit rules.

For example,

- in face recognition, an ML model learns features like eye spacing or facial shapes from labeled images to identify new faces.
- in spam email classification, it learns from examples of spam and non-spam emails to detect patterns like specific keywords.
- in weather prediction, ML analyzes historical data to forecast patterns, adapting to complexities that traditional programming cannot manage.

Below is an example of an ML program to classify student grades as pass or fail using a Logistic Regression model, showing how ML learns from data:

```
# ML program to classify student grades as pass or fail
import numpy as np # Library for numerical operations
from sklearn.linear_model import LogisticRegression # Import Logistic Regression model

# Generate sample data: scores and pass/fail labels (0 for fail, 1 for pass)
X = np.array([[i] for i in range(0, 101, 5)]) # Scores from 0 to 100 in steps of 5
y = np.array([1 if i >= 50 else 0 for i in range(0, 101, 5)]) # Labels: 1 for pass, 0 for fail

# Create and train the model
model = LogisticRegression() # Initialize Logistic Regression model
model.fit(X, y) # Train model on data (X: scores, y: labels)

# Predict for a new score
test_score = [[75]] # Test input
prediction = model.predict(test_score) # Predict label
print(f"Score 75 is {'Pass' if prediction[0] == 1 else 'Fail'}") # Output result
```

- *numpy* (imported as *np*) manages arrays and numerical operations, essential for ML data processing.
- The *sklearn.linear_model.LogisticRegression* module provides the Logistic Regression model, a classifier for binary tasks like pass/fail classification.
- The *fit* method trains the model by identifying patterns in the data, using X (input scores) and y (labels).
- The *predict* method applies learned patterns to new inputs for classification.
- Key Python ML libraries include *scikit-learn* (for models like Logistic Regression), *pandas* (for data manipulation), and *numpy* (for numerical operations).
- These libraries offer functions for data processing, model training, and evaluation, simplifying ML tasks.

Prompt engineering involves designing and refining input prompts to guide generative AI models, such as ChatGPT or Gemini, to produce desired outputs, like text or images. The quality of a response depends on factors like

- clarity (using precise language to avoid ambiguity)
- specificity (defining the task or context clearly)
- tone (specifying formal or informal style)
- constraints (setting limits like length or format)

For example,

- A vague prompt like “*Write about technology*” may produce a general response, while a specific prompt like “*Write a 5-line poem about technology in education for high school students*” generates a focused, relevant output.

Crafting effective prompts ensures accurate and relevant results from generative AI.

Exercise

Part 1: Traditional Programming vs AI/ML

Task 1A – Rule-Based Programming

- Write a Python program to check if a number is even or odd

Deliverables:

- Screenshot of your code and output.
- Explain the limitations of this approach.

Task 1B – ML-Based Classification

- Use the below code to simulate ML-based even/odd classification

Deliverables:

- Paste output
- Explain the difference from rule-based logic.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# Creating a dataset
data = {
    "value": [1, 2, 3, 4, 5, 6, 7, 8],
    "label": [1, 0, 1, 0, 1, 0, 1, 0] # 1=Odd, 0=Even
}
df = pd.DataFrame(data)

# Separate input (X) and output (y)
X = df[["value"]]
y = df["label"]

# Train the model
model = DecisionTreeClassifier()
model.fit(X, y)

# Predict
test_values = pd.DataFrame({"value": [10, 11, 13]})
print("Predicted labels (1=Odd, 0=Even):")
print(model.predict(test_values))
```

Part 2: Generative AI and Prompt Engineering

- Use ChatGPT or Gemini to generate:
 - A short story or poem
 - An AI-generated image
- Try prompts like:
 - Write a short 5-line poem about Artificial Intelligence in schools.
 - Describe a futuristic classroom in 2035 using vivid imagery.
 - "What are the ethical concerns of AI in Education?"

Deliverables:

- Paste 2 generated outputs
- Reflect on how prompt quality affects output.

• **Optional – Prompt Engineering:**

- Prompt: Explain artificial intelligence.
- Prompt: Explain AI to a 7-year-old using cartoon characters.

Deliverables::

- Compare the two outputs.